13 Pipe-Filter Architectural Style

In this chapter, we present and explain the Pipe-Filter architectural style and how to specify it in SysADL. We specify the style using the structural and behavioral viewpoints. We also present the pipeline architectural style as a substyle of Pipe-Filter. Finally, we illustrate the Pipe-Filter style and how to use it with our running example.

You will learn:

- what is a Pipe-Filter architectural style;
- what are its architectural elements, structure, and behavior;
- the pipeline substyle.

13.1 Conceptual Overview

In the Pipe-Filter style, components and connectors have a particular behavior and should be configured to allow a sequential processing of data.

Components, called “filters”, read streams of data on its inputs and produces streams of data on its outputs, typically applying a local transformation to each element of the input streams and incrementally computing the corresponding elements of the output streams.

Connectors, called “pipes”, serve as conduits for the streams, transmitting outputs of one filter to inputs of another.

We can use the pipe-filter style to model a part of a system that has a sequential dataflow that is transformed by filter components. The filter component acts as a filter to transform the flowing data. Each filter component has at least an *in* port (*Source*) or an *out* port (*Sink*). The connector acts as a pipe to conduct data from a sink in one component to a source in other component. Components are sequentially connected. ***Figure 1*** shows an overview of these elements specified as abstract components. It shows three filters connected by two pipes.

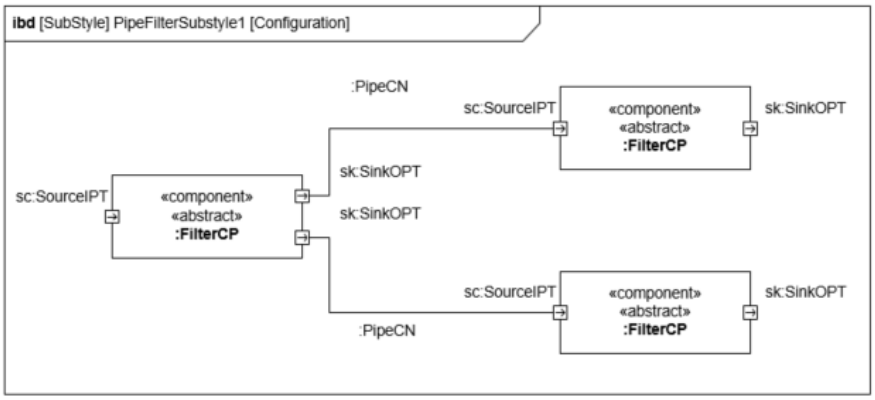

Figure 1. The Pipe-Filter Architectural Style

An example of the use of the pipe-filter style in the *RTC* System is an architecture with distributed temperature sensors with no direct communication with the controller. Instead, they are connected with their adjacent sensors in a sequence.

In this example, depicted in ***Figure 2***, we have composite temperature sensors composed of a sensing component and a transmitting component.

The *CompositeTempSensorCP* component has an internal temperature sensor and a component that sends temperature values that it receives in its in port. *s1*, *s2*, and *s3* components are the filters, and *c13* and *c23* connectors are the pipes in this architecture. The *s1* and *s2* components are connected to the s3 component. The *s3* component receives the temperature and sends it to the *outTemp* port.

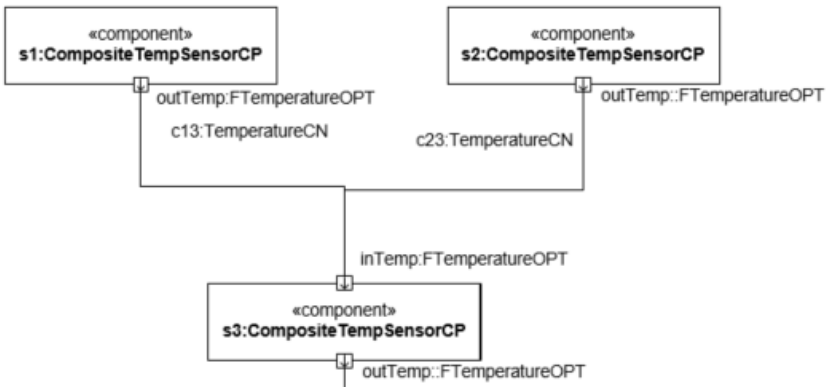

Figure 2. A Pipe-Filter example

13.2 Pipe-Filter Structural Viewpoint

We define the Pipe-Filter architectural style using a *bdd*, as illustrated in ***Figure 3***. The Pipe-Filter Architecture Style (*PipeAndFilterARCH*) is composed of at least 2 filter components (*FilterCP*). Each filter component must have at least one *in* port (*SourceIPT*) and one *out* port (*SinkOPT*). As a convention we named the *bdd* as *PipeAndFilterSCPD – SCPD* means *Style Component Definition*. All components must be connected by at least one filter.

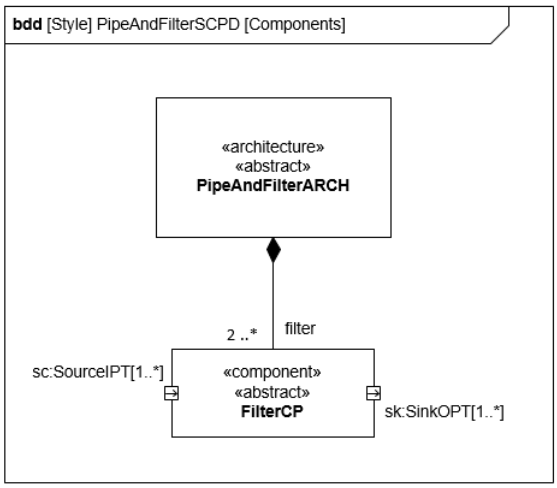

Figure 3. A Pipe-Filter definition

In the Pipe-Filter architectural style definition, we must also define the ports of a filter. ***Figure 4*** shows the ports definitions. The *SourceIPT* port receives values of any type. The *SinkOPT* port sends values of any type.

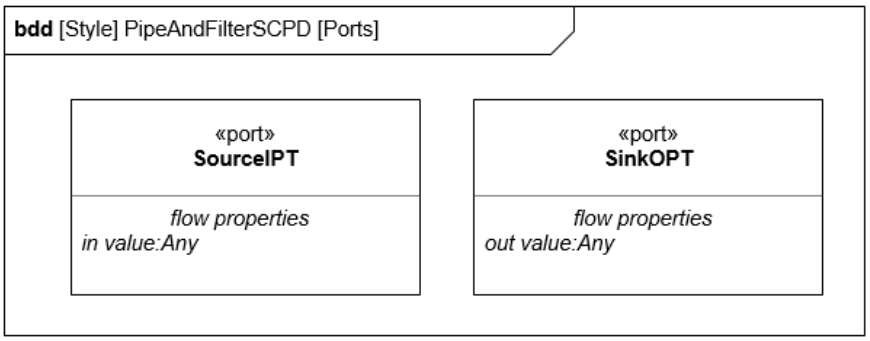

Figure 4. Pipe-Filter port definitions

In the Pipe-Filter architectural style definition, we must also define the pipe connector. ***Figure 5*** shows the connectors definitions of the style. The *PipeCN* connector has two participants: one *SourceOPT* port and one *SinkIPT* port. The *PipeCN* connector allows any type of data to flow from the *SourceOPT* port to the *SinkIPT* port.

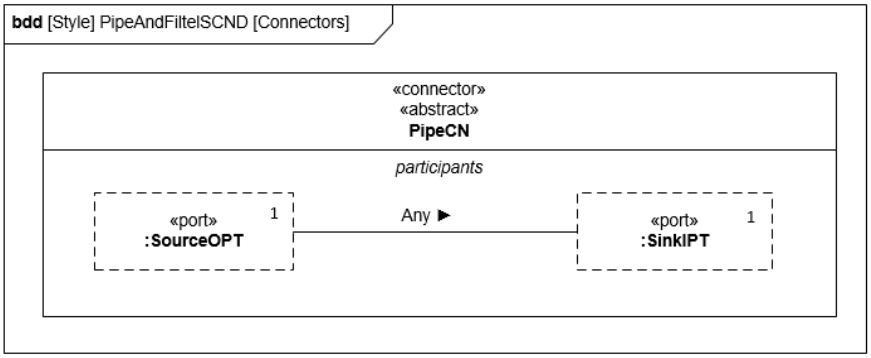

Figure 5. Pipe-Filter connectors definitions

13.3 Pipe-Filter Behavioral Viewpoint

We specify the behavior of the pipe connector using activity diagrams, as shown in ***Figure 6***. The connector sends a value directly from the input pin to the output pin.

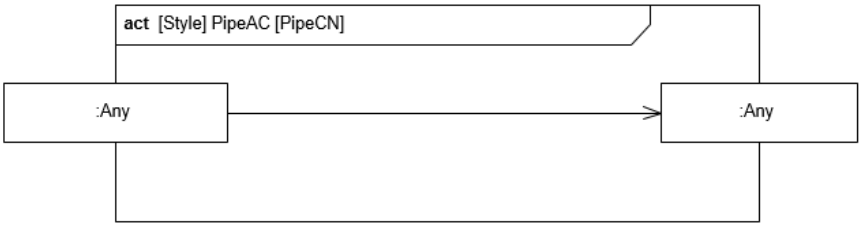

Figure 6. Filter connector behavior

We define the protocol of the ports using activity diagrams, shown in ***Figure 7***. The *SourceIPC* protocol states that the port repeatedly receives values of any type. The *SinkOPC* protocol states that the port repeatedly sends values of any type.

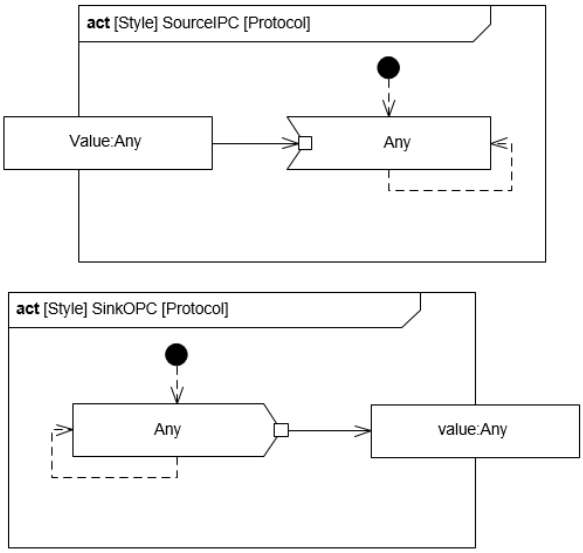

Figure 7. Port protocol behavior

13.4 The Pipeline Substyle

A Pipeline is a substyle of the Pipe-Filter style where all filters are sequentially connected. Each filter has at most one predecessor and one successor.

An example of the use of the pipeline style in the RTC System is an architecture with distributed temperature sensors with no direct communication with the controller. Instead, they are connected to their adjacent sensors in a sequence.

We can use the pipeline substyle to model a part of the system that has a sequential dataflow that is transformed by the filter component. The components are, thereby sequentially connected, as depicted in ***Figure 8***.

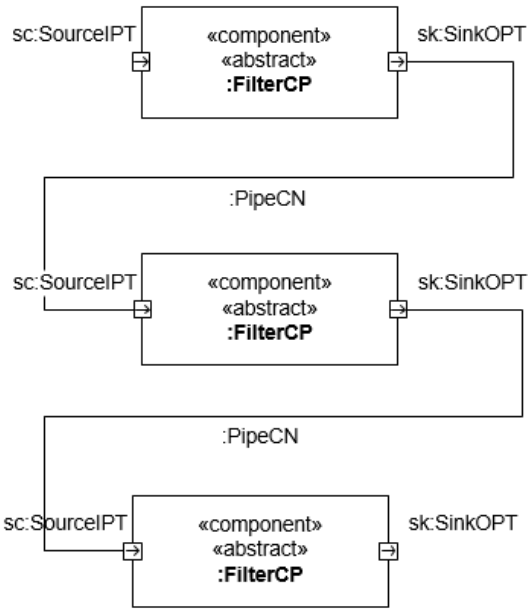

Figure 8. Pipeline style overview

In the example illustrated in ***Figure 9***, we can see composite temperature sensors in a pipeline architecture. The *CompositeTempSensorCP* component has an internal temperature sensor and a component that sends temperature values that it receives in its *in* port. The *s1*, *s2*, and *s3* components are filters, and the *c12* and *c23* connectors are pipes in this architecture; *s1* is connected to *s2*, and *s2* is connected to *s3*; *s2* and *s3* receive the temperature and send it to the *outTemp* port.

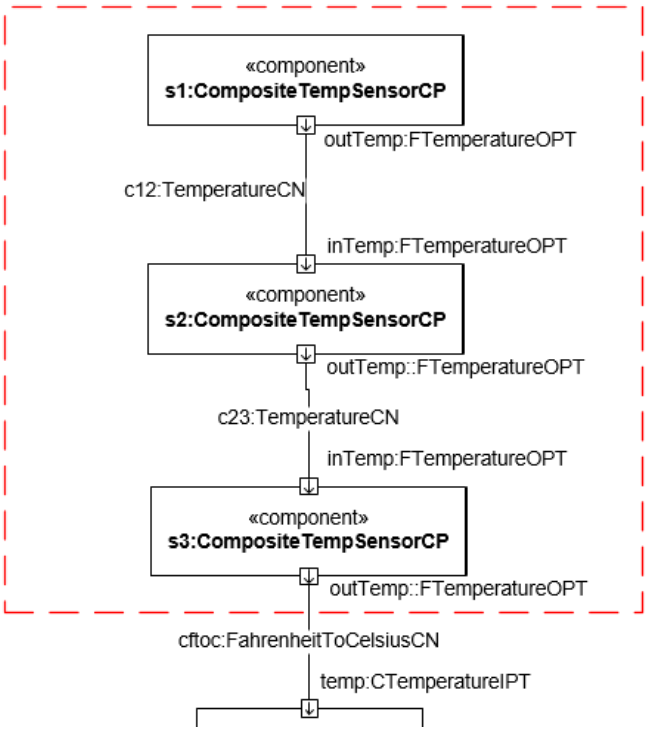

Figure 9. ***Pipeline style example***

13.5 Summary

In this chapter, you learned:

- the Pipe-Filter architectural style;
- the elements, structure and behavior of the Pipe-Filter style;
- the Pipeline substyle.

You learnt how to:

- apply the Pipe-Filter style in an architecture design.

13.6 For further reading

Clements, P.; Bachmann, L.; Garlan, D.; Ivers, J.; Little, R.; Merson, P.; Nord, R. Documenting Software Architecture: Views and Beyond. SEI Series in Software Engineering. (2003)

Buschmann, F., Meunier, R., Rohnert, H. Sommerlad, P., Michael Stal, M. Pattern-Oriented Software Architecture Volume 1: A System of Patterns. Vol. 1. Wiley (1996).

Vogel, O.; Arnold, I.; Chughtai, A.; Kehrer, T. Software Architecture: A Comprehensive Framework and Guide for Practitioners. Springer (2011)